

Importing the required models and packages

```
In [27]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

Loading the dataset

```
In [28]: # Loading the dataset
df = pd.read_csv('mymoviedb.csv', lineterminator = '\n')
```

Overview of the dataset

```
In [29]: # Overview of the dataset
df.head()
```

```
Out[29]:
```

	Release_Date	Title	Overview	Popularity	Vote_Count	Vote_Average	Original_Language	Genre	Poster_Url
0	2021-12-15	Spider-Man: No Way Home	Peter Parker is unmasked and no longer able to...	5083.954	8940	8.3	en	Action, Adventure, Science Fiction	https://image.tmdb.org/t/p/original/1g0dhYtq4i...
1	2022-03-01	The Batman	In his second year of fighting crime, Batman u...	3827.658	1151	8.1	en	Crime, Mystery, Thriller	https://image.tmdb.org/t/p/original/74xTEgt7R3...
2	2022-02-25	No Exit	Stranded at a rest stop in the mountains durin...	2618.087	122	6.3	en	Thriller	https://image.tmdb.org/t/p/original/vDHsLnOWKl...
3	2021-11-24	Encanto	The tale of an extraordinary family, the Madri...	2402.201	5076	7.7	en	Animation, Comedy, Family, Fantasy	https://image.tmdb.org/t/p/original/4j0PNHkMr5...
4	2021-12-22	The King's Man	As a collection of history's worst tyrants and...	1895.511	1793	7.0	en	Action, Adventure, Thriller, War	https://image.tmdb.org/t/p/original/aq4Pwv5Xeu...

```
In [30]: # Checking for any null values
df.info()
```

```
In [30]: # Checking for any null values
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9827 entries, 0 to 9826
Data columns (total 9 columns):
#   Column              Non-Null Count  Dtype
---  -
0   Release_Date        9827 non-null   object
1   Title               9827 non-null   object
2   Overview            9827 non-null   object
3   Popularity          9827 non-null   float64
4   Vote_Count          9827 non-null   int64
5   Vote_Average        9827 non-null   float64
6   Original_Language   9827 non-null   object
7   Genre               9827 non-null   object
8   Poster_Url          9827 non-null   object
dtypes: float64(2), int64(1), object(6)
memory usage: 691.1+ KB
```

```
In [31]: # Count of any duplicate values
df.duplicated().sum()
```

Out[31]: 0

```
In [32]: # Basic statistical analysis
df.describe().round(2)
```

Out[32]:

	Popularity	Vote_Count	Vote_Average
count	9827.00	9827.00	9827.00
mean	40.33	1392.81	6.44
std	108.87	2611.21	1.13
min	13.35	0.00	0.00
25%	16.13	146.00	5.90
50%	21.20	444.00	6.50
75%	35.19	1376.00	7.10
max	5083.95	31077.00	10.00

Exploration Summary

We have a dataframe of 9827 rows and 9 columns

Exploration Summary

We have a dataframe of 9827 rows and 9 columns

Our dataset does not contain any null or duplicate values

Release_date column converted from object to date time format

Changing the format of release date

```
In [33]: # changing format of release date from object to date&time
df['Release_Date'] = pd.to_datetime(df['Release_Date'])

print(df['Release_Date'].dtypes)

datetime64[ns]
```

```
In [34]: df['Release_Date'] = df['Release_Date'].dt.year

df['Release_Date'].dtypes
```

```
Out[34]: dtype('int32')
```

```
In [35]: # Removing unnecessary columns

cols = ['Overview', 'Original_Language', 'Poster_Url']
```

```
In [36]: df.drop(cols, axis = 1, inplace = True)
df.columns # removing the columns permanently
```

```
Out[36]: Index(['Release_Date', 'Title', 'Popularity', 'Vote_Count', 'Vote_Average',
               'Genre'],
              dtype='object')
```

```
In [37]: df.info()

<class 'pandas.core.frame.DataFrame'>
```

In [37]: df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9827 entries, 0 to 9826
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype  
---  -
0   Release_Date    9827 non-null   int32   
1   Title           9827 non-null   object  
2   Popularity      9827 non-null   float64  
3   Vote_Count      9827 non-null   int64   
4   Vote_Average    9827 non-null   float64  
5   Genre           9827 non-null   object  
dtypes: float64(2), int32(1), int64(1), object(2)
memory usage: 422.4+ KB
```

Labeling movies based on the vote_average

In [38]: *# Labeling movies based on the vote_average*

```
def categorise_col(df, col, labels):
    edges = [df[col].describe()['min'],
             df[col].describe()['25%'],
             df[col].describe()['50%'],
             df[col].describe()['75%'],
             df[col].describe()['max']]
    df[col] = pd.cut(df[col], edges, labels = labels, duplicates = 'drop')
    return df
```

In [39]: labels = ['Not recommended', 'Below average', 'average', 'Recommended']

categorise_col(df, 'Vote_Average', labels)

df['Vote_Average'].unique()

Out[39]: ['Recommended', 'Below average', 'average', 'Not recommended', NaN]
Categories (4, object): ['Not recommended' < 'Below average' < 'average' < 'Recommended']

In [40]: df.head()

Out[40]:

	Release_Date	Title	Popularity	Vote_Count	Vote_Average	Genre
0	2021	Spider-Man: No Way Home	5083.954	8940	Recommended	Action, Adventure, Science Fiction
1	2022	The Batman	3827.658	1151	Recommended	Crime, Mystery, Thriller

```
In [40]: df.head()
```

```
Out[40]:
```

	Release_Date	Title	Popularity	Vote_Count	Vote_Average	Genre
0	2021	Spider-Man: No Way Home	5083.954	8940	Recommended	Action, Adventure, Science Fiction
1	2022	The Batman	3827.658	1151	Recommended	Crime, Mystery, Thriller
2	2022	No Exit	2618.087	122	Below average	Thriller
3	2021	Encanto	2402.201	5076	Recommended	Animation, Comedy, Family, Fantasy
4	2021	The King's Man	1895.511	1793	average	Action, Adventure, Thriller, War

```
In [41]: df['Vote_Average'].value_counts()
```

```
Out[41]: Vote_Average
Not rcomended    2467
Recommended      2450
average          2412
Below average    2398
Name: count, dtype: int64
```

```
In [42]: # checking for any null values
df.isna().sum()
```

```
Out[42]: Release_Date    0
Title                  0
Popularity             0
Vote_Count             0
Vote_Average          100
Genre                  0
dtype: int64
```

```
In [43]: df.dropna(inplace = True)
```

Splitting the gener for better analysis

```
In [44]: ## We'd split genres inot list and then expaind our dataframe to have only one gener per row
```

```
df['Genre'] = df['Genre'].str.split(', ')
df = df.explode('Genre').reset_index(drop = True)
```

```
In [45]: df.info()
```

In [45]: df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 25552 entries, 0 to 25551
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Release_Date    25552 non-null  int32
1   Title           25552 non-null  object
2   Popularity      25552 non-null  float64
3   Vote_Count      25552 non-null  int64
4   Vote_Average    25552 non-null  category
5   Genre           25552 non-null  object
dtypes: category(1), float64(1), int32(1), int64(1), object(2)
memory usage: 923.6+ KB
```

In [46]: df.nunique()

```
Out[46]: Release_Date    100
Title                9415
Popularity           8088
Vote_Count           3265
Vote_Average         4
Genre                 19
dtype: int64
```

DATA VISUALIZATION

In [47]: sns.set_style("whitegrid")

Q. What is the most frequent genre of movies released on Netflix?

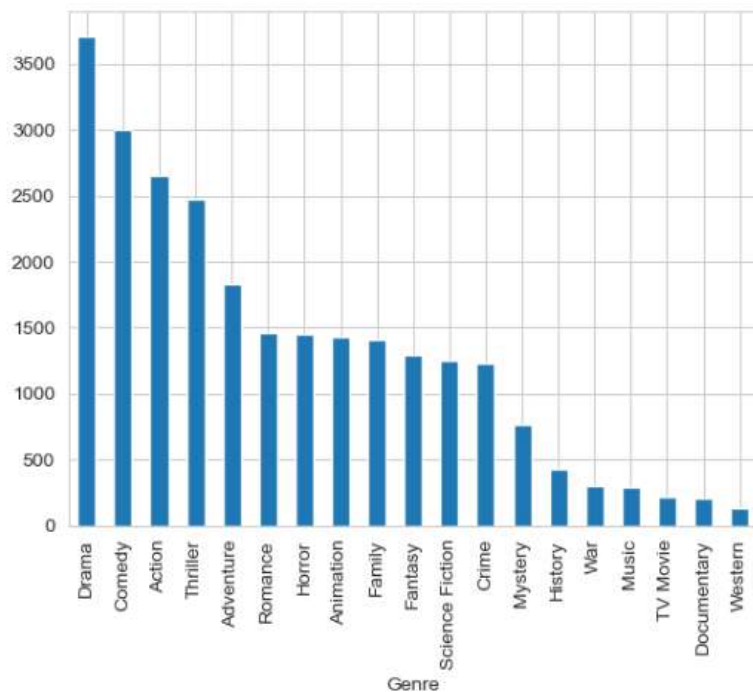
In [48]: df['Genre'].describe()

```
Out[48]: count      25552
unique         19
top           Drama
freq          3715
Name: Genre, dtype: object
```

In [49]: sns.catplot(y = 'Genre', data = df, kind = 'count', order = df['Genre'].value_counts().index,color = '#4287f5')

```
In [50]: categories = df['Genre']
categor = pd.Series(categories)
categor.value_counts().plot(kind = 'bar')
```

```
Out[50]: <Axes: xlabel='Genre'>
```



Q. What movie got the highest popularity? what's its genre?

```
In [51]: df[df['Popularity'] == df['Popularity'].max()]
```

```
Out[51]:
```

Release_Date	Title	Popularity	Vote_Count	Vote_Average	Genre
--------------	-------	------------	------------	--------------	-------

Q. What movie got the highest popularity? what's its genre?

```
In [51]: df[df['Popularity'] == df['Popularity'].max()]
```

```
Out[51]:
```

	Release_Date	Title	Popularity	Vote_Count	Vote_Average	Genre
0	2021	Spider-Man: No Way Home	5083.954	8940	Recommended	Action
1	2021	Spider-Man: No Way Home	5083.954	8940	Recommended	Adventure
2	2021	Spider-Man: No Way Home	5083.954	8940	Recommended	Science Fiction

Q. What movie got the lowest popularity? what's its genre?

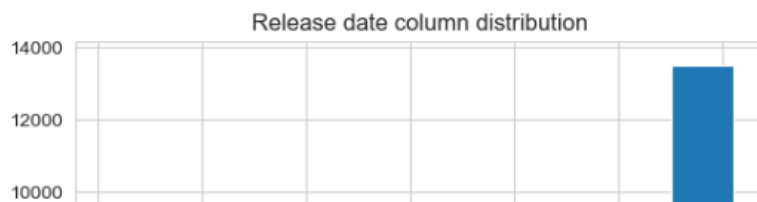
```
In [52]: df[df['Popularity'] == df['Popularity'].min()]
```

```
Out[52]:
```

	Release_Date	Title	Popularity	Vote_Count	Vote_Average	Genre
25546	2021	The United States vs. Billie Holiday	13.354	152	average	Music
25547	2021	The United States vs. Billie Holiday	13.354	152	average	Drama
25548	2021	The United States vs. Billie Holiday	13.354	152	average	History
25549	1984	Threads	13.354	186	Recommended	War
25550	1984	Threads	13.354	186	Recommended	Drama
25551	1984	Threads	13.354	186	Recommended	Science Fiction

Q. Which year has the most filmed movies?

```
In [53]: df['Release_Date'].hist()  
  
plt.title("Release date column distribution")  
plt.show()
```

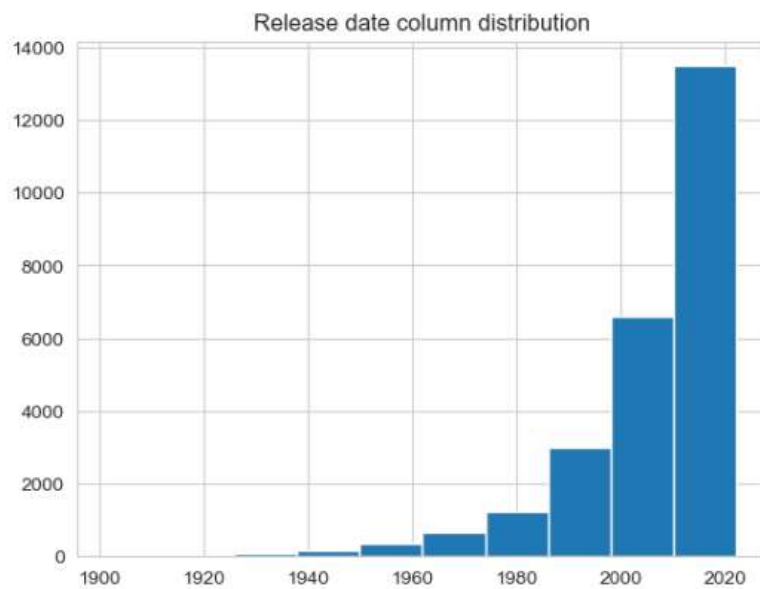


25546	2021	The United States vs. Billie Holiday	13.354	152	average	Music
25547	2021	The United States vs. Billie Holiday	13.354	152	average	Drama
25548	2021	The United States vs. Billie Holiday	13.354	152	average	History
25549	1984	Threads	13.354	186	Recommended	War
25550	1984	Threads	13.354	186	Recommended	Drama
25551	1984	Threads	13.354	186	Recommended	Science Fiction

Q. Which year has the most filmed movies?

```
In [53]: df['Release_Date'].hist()

plt.title("Release date column distribution")
plt.show()
```



```
In [ ]:
```